

Leant2: Lean-Native Program Synthesis from Types and Behavioral Specifications

Architecture, main algorithms, trust boundaries,
and compiler-checked feasibility experiments

September 17, 2026

Abstract

Leant2 should not be a transcription of a Haskell synthesizer into Lean. It should be a synthesis system whose authoritative objects are Lean expressions, dependent local contexts, universe constraints, and proof obligations. We propose a portfolio of cooperating search procedures: focused native term construction, demand-directed library composition, recursor and termination-schema synthesis, behavioral constraint propagation, and certified domain-specific search. A dependency-aware search state keeps program holes and their proof obligations together, so failure of a later obligation can revise an earlier computational choice. Immutable search recipes coexist with transactional Lean metavariable states; no second approximation of Lean’s type system is imposed on the general engine.

The design separates kernel-checked inhabitants, proved behavioral specifications, finite observations, grammar exhaustion, and genuine proofs of noninhabitation. It explains how to handle indexed inductives, dependent pairs, higher-rank arguments, noncumulative universes, local instances, recursive calls, and external solver proposals without confusing search heuristics with logical authority. Conditional completeness is stated only relative to an effective search grammar and fair scheduling. A practical evaluation and implementation plan makes broad coverage a measurable objective rather than an unsupported prediction.

Two accompanying Lean prototypes were remotely checked using Lean 4.34.0. One synthesizes dependent and polymorphic terms and fills the branches of a supplied indexed-vector recursion sketch. The other executes a finite counterexample-guided affine-program search and returns a proof of its infinite-domain specification. These experiments establish narrow implementation feasibility; they are not an implementation or a coverage benchmark of the complete proposed system.

Artifact status. Design proposal with checked experimental code. Full source, selected verification receipts, failed-attempt summaries, and reproduction tools accompany this article. No existing repository was modified.

Contents

1	Recommendation and scope	5
2	What to inherit, what to replace	5
2.1	Reviewed evidence and revision boundaries	5
2.2	The limitations are more specific than “Haskell cannot do Lean”	6
2.3	Relationship to existing synthesis research	6
3	The exact synthesis contract	7
3.1	A problem is larger than its displayed type	7
3.2	Four different kinds of constraint	7
3.3	Public frontend	8
4	Answers, refusals, and the meaning of completeness	8
4.1	A result algebra	8
4.2	Logical unprovability is not Lean negation	9
4.3	Coverage should be a vector, not a slogan	9
5	Architecture and ownership	10
5.1	One semantic core, several proposal mechanisms	10
5.2	Native expressions plus immutable recipes	10
5.3	Process and version boundaries	11
6	Native dependent search state	11
6.1	The state is an obligation graph	11
6.2	Why ordinary backtracking is insufficient	12
6.3	Transactional state and nontransactional budgets	12
6.4	Universes, binders, and equality	13
7	The native construction calculus	13
7.1	Bidirectional and focused organization	13
7.2	Head and spine synthesis	13
7.3	Unification policy	14
7.4	Cases and dependent elimination	14
7.5	Typeclass synthesis and explicit dictionaries	15
8	Demand-directed library and carrier discovery	15
8.1	An indexed inventory	15
8.2	Backward demand and scoped subproblems	16

8.3	Discovering hidden carriers	16
8.4	Provider and semantic restrictions	16
9	Scheduling, memoization, and relative completeness	17
9.1	A portfolio with a fair reference lane	17
9.2	Budgets must include proving the candidate	17
9.3	Cache keys and reuse rules	18
9.4	State-selection policy	18
10	Recursive programs as synthesized elimination schemas	18
10.1	The right search object is a schema, not unrestricted self-reference	18
10.2	Recursive-call capabilities	19
10.3	Algorithm for structural schema discovery	19
10.4	Indexed-vector map: a complete worked construction	20
10.5	Simultaneous program and proof synthesis	20
10.6	Accumulator introduction and invariant discovery	21
10.7	Well-founded, mutual, and nested recursion	21
11	Behavioral synthesis as construction, not only filtering	22
11.1	The behavioral loop	22
11.2	Logical decomposition and library contracts	22
11.3	Live examples and incomplete terms	23
11.4	Counterexample identity and replay	23
11.5	Proof services must not silently choose the program	23
12	Certified domain lanes and solver integration	24
12.1	A domain interface with a semantics	24
12.2	Suitable first domains	24
12.3	External SMT and synthesis engines	25
12.4	Typed equality saturation and optimization	25
13	Worked scenarios and boundaries	25
13.1	Dependent construction with a delayed witness	25
13.2	Higher-rank plumbing without an approximate rank calculus	26
13.3	Certified sorting	26
13.4	Quotients, extensional values, and effects	26
13.5	Constructive, classical, and executable modes	26

14 Acceptance, trust, and executable meaning	27
14.1 A small, explicit acceptance path	27
14.2 What a kernel proof does not certify	28
14.3 Hardened verification for untrusted extensions	28
14.4 No evidence laundering through presentation or caches	28
15 Compiler-checked experiments	28
15.1 Environment and reproducibility	28
15.2 Native dependent search	29
15.3 A finite certifying behavioral lane	30
15.4 Failed attempts and prototype shortcomings	31
16 Evaluation: making practical coverage measurable	31
16.1 A benchmark with separate objectives	31
16.2 Preventing target leakage	32
16.3 Baselines and ablations	32
16.4 Adversarial acceptance tests	32
17 Implementation roadmap and acceptance gates	33
17.1 Milestone 1: a trustworthy native core	33
17.2 Milestone 2: scalable compositional search	33
17.3 Milestone 3: dependent recursion	33
17.4 Milestone 4: certifying behavior	33
17.5 Milestone 5: broader algorithmic coverage	33
17.6 Priority risks	33
18 Conclusion	34
A Implementation interfaces and invariants	35
B The checked continuation-search implementation	36
C The checked behavioral certificate boundary	37
D Reproduction and artifact map	37
E Status ledger	39
References	39

1 Recommendation and scope

The recommended system is a *Lean-native, proof-producing program synthesizer with several cooperating search lanes*. Its organizing unit is not a translated simple type and not an isolated tactic goal. It is an exact Lean synthesis problem together with a graph of mutually dependent computational and logical obligations. All lanes propose terms or refinements of that graph. A small acceptance path checks the resulting declarations and behavioral proofs in Lean.

Three decisions are fundamental. First, Lean’s own expressions and contextual typing information remain authoritative throughout the general search. Second, recursion and behavioral reasoning are construction mechanisms, not expensive filters attached only after complete programs have been enumerated. Third, every negative answer identifies what was actually ruled out. A bounded heuristic miss is never a proof that the requested Lean type is empty.

The goal is substantial coverage of ordinary programming and verification tasks: polymorphic adapters; products, sums, structures and classes; finite and recursive containers; indexed data; proofs accompanying computed values; compositions of library operations; and recursive programs characterized by equations or relational specifications. Unrestricted theorem proving, unrestricted higher-order unification, arbitrary termination arguments, and arbitrary semantic program equivalence are not assumed decidable. Difficult cases remain searchable through increasingly powerful lanes and user-supplied sketches, without inventing a completeness claim.

What the article contributes. The main contribution is a concrete synthesis architecture: a native dependent-state model, a continuation-correct construction algorithm, demand-directed provider and carrier discovery, recursor-based program/proof co-synthesis, a certifying behavioral loop, and explicit interfaces between these mechanisms. The accompanying code tests two of those ideas in small settings. The architecture as a whole, its scheduler, large-scale retrieval system, and recursive-scheme discovery are proposals, not implemented features of the artifact.

2 What to inherit, what to replace

2.1 Reviewed evidence and revision boundaries

This design uses selected source and documentation from Leant commit

6bf05ad78c467989e68290f2d08bbbed40802d485

and standalone Djex commit

e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5.

Both are September 14, 2026 snapshots. The reviewed Leant documentation separately identifies its vendored Djex dependency as `e237e866`; that is not the same revision as the standalone Djex main snapshot. The comparison is architectural, not a reproduction of the predecessors’ complete test suites. Relevant evidence is the actual fragment translator, the synthesis and behavioral guides, Djex’s architecture guide, the existing Lean-rewrite analysis, and the latest reviewed indexing diagnostic [1, 2, 3, 4, 5, 6].

The existing rewrite analysis already makes the case for eliminating cross-language representation crossings and retaining project-local process isolation. It also recommends keeping an explicit neutral

search representation, with Lean metaprogramming concentrated at preparation and lowering. Leant2 should retain the operational insight but change that last choice for its dependent lane: native metavariables and dependent contexts are valuable *inside* search. Determinism and serialization should come from immutable recipes and controlled state ownership, not from translating the whole problem into a smaller type language [5].

2.2 The limitations are more specific than “Haskell cannot do Lean”

The predecessors have already developed substantial polymorphic and higher-order support. It would be inaccurate to describe them as merely monomorphic arrow-type inhabitation engines. The concrete mismatch is that the Lean fragment translator must encode selected distinctions explicitly, and some structures, notably term-indexed applications, are still handled opaquely rather than decomposed as arbitrary dependent Lean types [1]. A native engine removes that mandatory projection. It does not automatically invent a good search order, an invariant, or a termination argument.

The reviewed indexing diagnostic is especially instructive. Its original bounded Haskell query rejects all 256 observed candidates, although useful function-valued fold states and the integer-case step are individually available. The isolated step is found at candidate 5; adding surrounding context changes the observed successful position to 219. These are diagnostic positions, not an internal trace of the original recursive construction. The same report records continued bounded misses for the original simultaneous-universe fixture [6]. The architectural lesson is that representation fidelity and compositional search quality are separate problems.

Table 1: Successor decisions, not a promise to preserve every predecessor interface.

Concern	Preserve	Replace or extend
Typing authority	Exact candidate checking and source identity	Native <code>Expr</code> , contexts and levels instead of a mandatory approximate type projection
Search diversity	Distinct logical and heuristic search strategies	Dependent construction, recursion schemas, behavioral feedback and fair continuations
Behavior	Passed/falsified/inconclusive distinction	Obligations propagated into partial programs, with certified domain lanes
Evidence	Candidate-specific provenance and bounded claims	Direct expression/proof artifacts, rather than text as the semantic bridge
Runtime	Project-local worker, cancellation and recovery	Lean controller and worker; no Haskell frontend compatibility layer

2.3 Relationship to existing synthesis research

Type-directed search and refinement-based program construction are established ideas. Synquid demonstrates the value of decomposing a refinement specification during recursive synthesis; constraint-based type-directed synthesis develops the interaction between typing and synthesis constraints; live bidirectional evaluation explains how examples can constrain incomplete programs [12, 13, 15]. Leant2 should adapt these mechanisms to an exact Lean environment rather than claim that dependent program synthesis is a new field.

Aesop is particularly relevant as Lean-native proof-search infrastructure with shared metavariables [17]. It can be a proof-obligation service and a source of engineering patterns. It is not, by itself, a

replacement for computational witness enumeration, example propagation, recursion-scheme discovery, or candidate-quality policies. Similarly, a future Forge integration should be an optional implementation of a proof-service interface, not a dependency that defines Leant2’s synthesis semantics.

3 The exact synthesis contract

3.1 A problem is larger than its displayed type

Definition 3.1 (Frozen synthesis problem). *A problem is a tuple*

$$Q = (\mathcal{E}, \Gamma, \mathcal{U}, T, \Phi, \mathcal{O}, \mathcal{G}, \mathcal{A}, \mathcal{B}).$$

Here $\mathcal{E}$ is the exact Lean environment; Γ is the ordered dependent local context, including local definitions and instances; $\mathcal{U}$ records universe parameters and constraints; T is the requested type; $\Phi(t)$ is an optional mandatory proposition about a candidate; $\mathcal{O}$ is a collection of observations; $\mathcal{G}$ is the admitted construction grammar and provider inventory; $\mathcal{A}$ is the trust and execution policy; and $\mathcal{B}$ is the resource policy.

The successful judgment is

$$\mathcal{E}; \Gamma \vdash t : T, \quad \mathcal{E}; \Gamma \vdash p : \Phi(t) \quad \text{when a mandatory specification is present.}$$

The type and specification are elaborated once in the user’s context and then frozen. Elaborating an unknown name must not silently create an auto-implicit parameter in a behavioral clause. Likewise, an engine must not introduce `Inhabited A`, replace an input by a defaulted variant, specialize two independent universe parameters to one, or weaken a postcondition merely to make synthesis succeed.

It is tempting to treat every query as a subtype-inhabitation problem. That is useful for many data-valued targets, but the implementation should retain separate term and proof obligations. It handles arbitrary requested sorts, avoids accidental changes to elimination behavior, and makes it possible to rank executable code separately from its certificate. The obligations share the same candidate metavariable, so this separation is operational rather than semantic.

3.2 Four different kinds of constraint

A *logical requirement* is an exact Lean proposition, for example

$$\forall xs, \text{Perm}(f(xs), xs) \wedge \text{Sorted}(f(xs)).$$

Acceptance requires a proof of that proposition. A *finite observation* might instead be $f([3, 1]) = [1, 3]$. A finite conjunction of such observations establishes exactly that conjunction, not a universal sorting theorem. For a genuinely finite input type, exhaustive checking can establish a universal theorem only when the enumeration’s coverage and the evaluator’s correctness are also justified.

A *construction restriction* controls which implementations may be proposed: no calls to a particular library function, at most one structural recursion, or use only an explicit provider set. It is a grammar predicate, not a mathematical property of all equivalent functions. Finally, an *operational objective* concerns runtime, allocation, or an asymptotic cost model. A smaller expression is not automatically faster. Such objectives must be either proved relative to an explicit cost semantics or reported as empirical measurements.

The user should be able to supply all four independently. This prevents an example-only mode from looking certified and prevents a library-reuse benchmark from being mistaken for synthesis from primitives.

3.3 Public frontend

The primary frontend should be a Lean library and tactic, with command and editor layers around the same request API. The following is *proposed syntax*, not syntax implemented by the prototype:

```
def adapter : (A -> B) -> List A -> List B := by
  leant2 using [List.map]

#synth transform : List Nat -> List Nat
  satisfying (fun f => forall xs, f xs = xs.reverse)
  excluding [List.reverse]

-- A sketch fixes the algorithmic skeleton, not the hole contents.
def vectorMapSketch ... := by
  leant2 completing ...
```

The API should return structured events and final statuses. Editor support can display the current partial term, the obligation blocking it, a concrete rejected example, and the cost frontier. A **synth?**-style command should insert ordinary Lean code plus any required theorem, not a fragile transcript of the entire search. A REPL is useful but should not be the semantic owner of synthesis.

4 Answers, refusals, and the meaning of completeness

4.1 A result algebra

The result type should distinguish at least the following outcomes.

Certified candidate

An exact term, its checked type, mandatory specification proof if any, explicit axiom dependencies, and execution-policy assessment.

Observed candidate

A type-checked term with precisely named finite observations. This result cannot satisfy a query that required a universal proof.

Certified empty

A Lean proof of $T \rightarrow \text{False}$, or of $\neg\exists t : T, \Phi(t)$, under the frozen context. This is the strongest negative result.

Grammar or fragment exhausted

An exhausted finite grammar or a certificate that no derivation exists in a named formal fragment, with its exact scope and coverage statement. It need not imply that the Lean target is empty.

Unknown

A structured reason: bound reached, unresolved unification, missing provider, unsupported elimination, proof timeout, inconclusive solver result, or interrupted worker. Remaining continuations should be resumable when possible.

A mode can request only the first result, the best result within a bound, or a stream of diverse candidates. None of these modes should change what certification means.

4.2 Logical unprovability is not Lean negation

A complete intuitionistic propositional search procedure is useful. Its failure, however, cannot generically be converted into a Lean proof of the negation of the interpreted goal. For instance, an intuitionistic propositional calculus does not derive excluded middle uniformly, while Lean with its classical principles can prove it. A Kripke countermodel concerns derivability in the specified calculus; it does not make every interpretation of the formula false inside the current Lean environment.

There are three safe routes to a stronger negative statement. One may construct an explicit Lean contradiction from a hypothetical inhabitant. One may exhaust a genuinely finite domain with a checked coverage theorem. Or one may use a formally proved reflection/coverage theorem whose conclusion is exactly the desired Lean noninhabitation claim. Merely declaring a source projection “complete enough” is not the same as providing such a theorem.

The prototype includes an elementary example of the first route:

```
theorem uniformEmpty : (forall A : Type, A) -> False := by
  intro f
  exact Empty.elim (f Empty)
```

This is a genuine negative certificate. In contrast, the prototype’s expected failures on **False** and on an impossible natural-number subtype are tests of a bounded tactic, not machine-checked completeness theorems about that tactic.

4.3 Coverage should be a vector, not a slogan

A practical coverage record should have independent axes for syntax, construction rules, logical certificates, execution, and resource bounds. A system might understand an indexed type exactly, construct its constructors, but have no recursive elimination scheme yet. It should say so. Likewise, it may synthesize an inhabitant of a higher-rank type without finding the intended behavior under the current constraints.

The objective “a large fraction of types occurring in practice” should therefore be operationalized by a task distribution and a held-out benchmark, not by a type-theoretic universality claim. Section 16 gives a concrete evaluation protocol.

5 Architecture and ownership

5.1 One semantic core, several proposal mechanisms

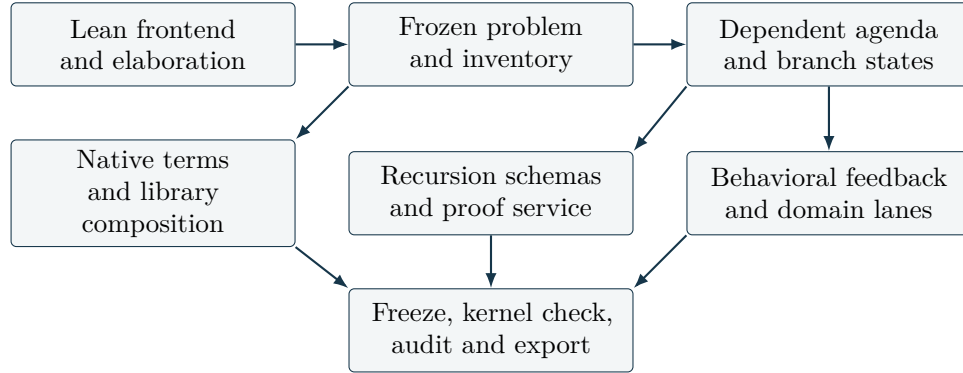


Figure 1: Proposal lanes share one frozen problem and acceptance boundary. Behavioral observations also feed back into the agenda; the diagram suppresses those return arrows.

A practical module layout is:

Leant2/Frontend	syntax, tactic, command, editor events
Leant2/Problem	freezing, scopes, policies, identities
Leant2/Native	elaboration adapter, native state, unification
Leant2/Search	agenda, recipes, continuations, budgets
Leant2/Inventory	declaration indices, retrieval, local providers
Leant2/Construct	introductions, spines, cases, transports
Leant2/Recursion	motives, call capabilities, termination schemas
Leant2/Behavior	observations, obligations, replay, counterexamples
Leant2/Domains	certified finite languages and solver adapters
Leant2/ProofService	Lean tactics and optional external integrations
Leant2/Accept	kernel validation, axiom and execution audit
Leant2/Worker	project process, protocol, cancellation, recovery

These boundaries are about ownership, not necessarily separate processes. Search modules can be highly heuristic and even contain partial metaprograms. The output is accepted because it survives the checker, not because every line of the search engine has been formally verified.

5.2 Native expressions plus immutable recipes

The general engine owns actual `Expr`, `Level`, `FVarId`, `MVarId`, `LocalContext`, and environment information. It should not define a replacement “Lean type” datatype with a hand-maintained approximation of universes, applications, or dependent binders.

Nevertheless, native mutable state is a poor durable file format. Every search action should also have an immutable recipe: introduce a binder, apply a named provider occurrence with these arguments, split this local, introduce this recursor motive, ask this proof service with this budget. Recipes refer to stable problem-local binder indices and declaration identities, not to raw metavariable numbers from another worker. A recipe can be replayed against the frozen problem to reconstruct a branch; checked native snapshots accelerate execution within one worker.

Small reified languages remain desirable in *domain-specific* lanes. An affine arithmetic syntax or a finite Boolean grammar can have a clean semantics and proved checker. Such a language is an optimization with a stated domain, not an ontology through which every Lean target must pass.

5.3 Process and version boundaries

The controller and worker can both be implemented in Lean. The worker should run under the target project’s pinned Lake environment. This avoids loading incompatible compiled modules into a globally installed process and permits killing a runaway elaboration without losing durable session history. The existing rewrite analysis identifies these reasons for retaining a worker boundary [5].

A worker owns an environment epoch. Candidate handles, native snapshots, memo entries, and proof-service responses are valid only in that epoch. Undo, environment replacement, or worker death invalidates them. Durable state consists of the exact source dependencies, problem description, settings, and replay recipes. An in-memory `Expr` from a retired worker is not an accepted cross-version certificate.

The baseline requires no Haskell component. Optional SMT executables do not alter that implementation choice. Strong operating-system guarantees, such as race-resistant descriptor-bound executable launch, should be specified separately; rewriting process code in Lean does not automatically preserve every security property of the predecessors’ platform shim.

6 Native dependent search state

6.1 The state is an obligation graph

A branch state should contain a native metavariable context, a root term hole, pending goals, universe constraints, delayed elaboration obligations, recursive-call permissions, behavioral observations, and a replay recipe. A useful conceptual representation is

$$S = (M, r, H, D, U, C, R, O, \rho),$$

where M is the branch’s native metavariable state, r is the root expression, H is the set of holes, D is their dependency graph, U is the universe store, C contains delayed constraints, R contains recursive-call capabilities, O contains observations, and ρ is the immutable recipe.

An edge $h_i \rightsquigarrow h_j$ means that assigning h_i may change the type, context, specification, or observation of h_j . Constructor application normally creates a dependent *hyperedge*, not independent child problems. For

$$\Sigma(x : A), B(x),$$

choosing x changes the second goal. For a subtype, choosing the witness changes a proof obligation. For a generic provider, choosing its type parameter changes the types of later ordinary and instance arguments. Branch identity must include all these shared assignments.

Invariant 6.1 (Branch ownership). *Every unassigned metavariable belongs either to the frozen surrounding problem or to one explicitly owned branch. A branch may not mutate frozen ambient metavariables, publish another branch’s free locals, or reuse an assignment whose context embedding has not been checked.*

When invoked in an unfinished Lean proof, the tactic cannot simply assume that every surrounding metavariable is its own. Preparation should identify the permitted assignment set. Read-only ambient holes can be abstracted as parameters or treated as rigid constraints. A successful proposal is then integrated transactionally into the caller’s proof state.

6.2 Why ordinary backtracking is insufficient

Consider

```
{n : Nat // n = 1}
```

A constructor-first search may try $n = 0$. That inhabits the witness type `Nat`; the later proof goal $0 = 1$ fails. If the engine permanently commits the first witness before searching its sibling proof, it misses the solution $n = 1$. The same error occurs with dependent dictionary arguments, indexed constructors, and behavioral proofs about an already synthesized function.

The correct unit of backtracking is the *whole continuation*, including the siblings whose types or specifications depend on a choice. The following schematic algorithm captures this requirement:

```
solve(state, goals, budget):
  remove goals already assigned by shared substitutions
  if no goals remain: return completed state
  g := selectGoal(goals, dependencyGraph)
  for refinement in fairRefinements(g, state):
    childState := restoreOrFork(state)
    children := applyRefinement(refinement, childState)
    answer := solve(childState, children + remainingGoals, budget')
    if answer succeeds: yield answer
    otherwise resume another refinement of g
```

The actual prototype uses this continuation structure with `Meta.saveState` and the saved state’s `restore` method. Its `children ++ rest` is not an incidental implementation detail: it is what allows a failed dependent sibling to revise an earlier witness. A production implementation should retain multiple successful continuations, not just the first one.

6.3 Transactional state and nontransactional budgets

Saving and restoring `MetaM` state is appropriate for native construction. When a step invokes a tactic or term elaborator, the adapter must also account for the elaborator’s state: synthetic metavariables, pending instances, messages, and any environment changes. It is unsafe to assume that a `Meta.SavedState` automatically snapshots every layer of an arbitrary plugin’s execution.

Resource accounting is deliberately *not* rolled back. Failed branches have consumed time, allocations, and search work. The global budget ledger lives outside logical branch snapshots. Cancellation and heartbeat exhaustion are interruption events, not ordinary failed alternatives to be swallowed and retried indefinitely. The small prototype catches exceptions uniformly for simplicity; production code must classify them and propagate nonrecoverable interruptions.

Parallelism should initially operate on immutable recipes or separate worker states. Two tasks must not concurrently mutate one shared native metavariable context. A completed task returns a closed proposal plus its provenance, or a replayable branch refinement; merging a raw set of assignments from unrelated snapshots is not supported.

6.4 Universes, binders, and equality

Lean’s universe levels are expressions, not Haskell kinds. The search must preserve rigid parameters and use fresh level metavariables for each new provider occurrence. **Type** is predicative and Lean’s universes are noncumulative; **Prop** has different quantification rules. A native implementation therefore uses Lean’s level constraints and requests explicit lifting constructions when needed, rather than silently coercing types upward [7].

Binder information is retained exactly: explicit, implicit, strict implicit, and instance implicit binders affect elaboration and reconstruction. Local instances are values with identities, not just a multiset of class names. Two dictionaries of the same class type can contain different operations. A synthesis rule must preserve the dictionary actually selected for that provider occurrence.

Definitional equality is handled by controlled normalization and native unification. Propositional equality requires a proof and, where necessary, an explicit transport such as an equality recursor. For example, converting a vector of length $n + m$ to one of length $m + n$ is not generally solved by pretending the indices are definitionally identical. The engine can prove the index equality and build a transport, with a cost that makes gratuitous casts unattractive.

Normalization needs a transparency policy and a work budget. Eagerly unfolding every definition is neither necessary nor efficient. Try weak-head reduction first; unfold selected reducible wrappers on demand; record when a step relied on a stronger transparency setting. Final checking is against the original target, not against an independently invented normal form.

7 The native construction calculus

7.1 Bidirectional and focused organization

The core combines goal-directed checking with type synthesis for applications. Introduction rules exploit the expected type; elimination rules exploit available heads and their telescopes. This is the same broad reason that type-directed synthesis can avoid untyped enumeration [13, 14]. It does not require a complete decision procedure for dependent types.

For a dependent function goal, introduce a local and continue in its extended context:

$$\frac{\Gamma, x : A \vdash b : B(x)}{\Gamma \vdash (\lambda x. b) : (x : A) \rightarrow B(x)}.$$

For a constructor c with dependent telescope Δ_c and result $I(\vec{p}, \vec{i})$, instantiate the telescope, unify the result with the expected family, and retain every remaining argument obligation. The fields are not flattened into an untyped product.

A focused lane prioritizes invertible decompositions and delays branching choices. The word “safe” is relative to a calculus and admissible grammar: a transformation that preserves inhabitation may still change cost, syntax restrictions, or the best executable representation. The system should not silently turn a proof-theoretic normalization principle into a claim of minimum program size.

7.2 Head and spine synthesis

For an elimination candidate h , the engine traverses its exact Pi telescope. It creates argument metavariables in order, substitutes them into later binder types, and attempts to match the residual result against the expected type. It then schedules unresolved arguments and side conditions. A

single provider may therefore induce data holes, type holes, proof holes, instance holes, and universe obligations.

```

expandHead(h, expected, state):
  freshen provider universes and make this occurrence explicit
  ty := inferType(h)
  args := []
  while ty exposes an admissible Pi binder:
    a := freshGoal(domain(ty), exactContext, binderInfo(ty))
    args.append(a)
    ty := instantiateBody(ty, a)
  offer a partial application when its type can meet expected
  constrain ty definitionally equal to expected
  retain unresolved arguments, instances and level constraints
  emit one shared branch, not separately solved arguments

```

Partial applications matter. A function-valued fold step may be an already available local function, rather than a lambda that is expanded and searched from scratch. Conversely, a higher-rank argument may need to be constructed by opening a local type binder inside that argument. Both cases arise naturally from the native telescope, without choosing a global fixed rank bound for the whole language.

Argument metavariables should not all be solved greedily. An implicit type argument may be fixed by the result; a proof may constrain a witness; a typeclass goal may remain underdetermined until a later index is chosen. Native unification is a mechanism for recording those constraints, not an obligation to guess immediately.

7.3 Unification policy

The first implementation should use Lean’s native machinery for routine definitional equality and a disciplined higher-order-pattern-oriented policy for search-generated holes. Flexible equations that do not resolve should become explicit delayed obligations. Retrying after a relevant assignment can solve many practical cases without a general higher-order-unification algorithm.

More ambitious alternatives, such as bounded imitation/projection choices or synthesis of type families, belong in a separate expensive lane. They must have explicit costs and resumable continuations. A failed or timed-out unification attempt gives no authority to claim the branch is impossible unless the failure is a genuine rigid contradiction. In particular, a suspended flexible equation is not equivalent to an inconsistent equation.

This policy preserves a useful distinction: the system can *represent* an arbitrary elaborated Lean target while admitting only selected ways to resolve its unknowns. A supported representation does not promise that every metavariable equation will be solved automatically.

7.4 Cases and dependent elimination

Case splitting should use Lean’s actual inductive and recursor information. The engine identifies the major premise, parameters, indices, motive, and constructor branches through native metadata and elaboration. It must generalize or transport dependent local hypotheses as required by the elimination, rather than treating a split as a substitution in a simple sum type.

Indexed families benefit immediately. A vector at index zero and one at successor index have different admissible constructor shapes. An impossible branch is discharged by a checked index

contradiction or no-confusion argument. A branch is not removed merely because a custom arithmetic heuristic predicts that its indices cannot match.

Elimination from propositions requires particular care. A proof of **False** eliminates into arbitrary sorts. A proof of **Nonempty A** is not, in the constructive computational mode, a freely extractable **A**. Likewise, **Exists** and a data-valued dependent pair are not interchangeable. Native recursor validation enforces the relevant distinction [8]. The prototype includes a rejection control for the attempted computational extraction from **Nonempty**.

Useful split candidates include a scrutinee demanded by an observation, a local inductive whose constructors expose a needed value, or a decidable proposition whose two branches have distinct residual specifications. Repeated irrelevant splitting should be penalized and bounded, not optimized away by an unsound assertion that it can never help.

7.5 Typeclass synthesis and explicit dictionaries

A production typeclass lane should first try normal Lean instance synthesis in the frozen local context. It should also retain named local dictionary values as explicit providers when the query or a provider application selects them. Backtracking must include instance assignments if computational behavior depends on the choice.

Do not cache a successful method application under only the class name and argument types. The key must identify its dictionary payload or a verified equivalent context embedding. Do not manufacture an extra global instance to solve a local hole: that changes the environment and can affect unrelated elaboration. Temporary instance exposure, when needed, belongs in a branch-local context and must be reproducible.

The included **fromInstance** test only extracts a field from a supplied class value by structural search. It is not evidence that general instance synthesis, overlapping instances, or superclass search has been implemented.

8 Demand-directed library and carrier discovery

8.1 An indexed inventory

Searching every declaration for every goal will not scale. Build persistent indexes from declaration types, not from pretty-printed names. A provider entry records its declaration identity, universe parameters, full telescope, terminal result shape, constructor or projection role, computational policy, and any supplied specification lemmas.

A discrimination-tree-style index over normalized result shapes is a good first filter. Rigid constants, family heads, and argument positions guide retrieval; flexible positions are wildcards. Every retrieved candidate still goes through native instantiation and checking. The index may have false positives. If it has intentional false negatives, a wider fallback lane must remain available and the restricted lane must not claim completeness.

Local declarations deserve a separate small index. Projection saturation can expose structure fields; bounded forward application can derive useful intermediate values. A typed shared DAG records the derivations and their exact contexts, avoiding repeated recomputation while keeping the expressions as authoritative objects.

8.2 Backward demand and scoped subproblems

A demand graph is built backward from target holes and specification observations. If a needed result has shape C , a provider ending in C induces demands for its arguments. Those demands in turn identify providers for $A \rightarrow B$, a dictionary, an index equality, or a recursive subresult. Bounded forward facts from the context meet this backward graph.

For each demand, compute a *candidate context slice*: locals in its type, specification, and pending constraints, together with a bounded predecessor closure through usable providers. Search that slice as a high-priority lane. Keep the full-context lane as fallback. A slice can improve ordering dramatically, but excluding a local merely because its type does not occur syntactically in the goal is not a proof that it is irrelevant.

A solution found in a slice is transported into the larger context using an explicit weakening or renaming map and checked again. Cached subsolutions are recipes parameterized over an abstracted telescope. They are not bare expressions containing stale free-variable identifiers. Negative results from the smaller context cannot be generalized to the larger one.

8.3 Discovering hidden carriers

For polymorphic folds and Church encodings, the right accumulator type is often absent from the outer target. Carrier synthesis should be demand-driven. Candidate carriers begin with result and argument types, products of demanded components, functions from a demanded control input to a result, and small indexed families already present in the environment. More complex native type expressions enter through increasing-cost enumeration.

Consider a Church list whose fold result can be chosen at `Type u`. To implement defaulted integer indexing, a useful carrier is `Int -> A`. The fold's base then has type `Int -> A`; its step has type

```
A -> (Int -> A) -> Int -> A
```

rather than simply `A -> A -> A`. The outer demand to consume an integer index suggests the function carrier. A branch-local specification distinguishes negative, zero, and positive inputs and makes the appropriate integer case operation relevant.

This design targets the kind of compositional ordering problem in the predecessors' diagnostic [6]. It does not establish that the original fixture is solved. The artifact has no end-to-end reproduction of that query, and its simpler two-universe test is not substituted for the original two-box signature.

8.4 Provider and semantic restrictions

The default inventory should be transparent to the user. A result should say whether it reused a complete library implementation, composed smaller operations, or filled a recursive sketch. Benchmark mode must enforce explicit exclusions: a hidden oracle definition, a private alias of the target, or a theorem that trivially reveals the target body can otherwise make apparent synthesis success meaningless.

Usage preferences are ranking features, not default logical constraints. Functions that ignore an argument can be perfectly correct. Conversely, mentioning every argument does not establish the intended behavior. A user may request a syntactic usage restriction, but it must be reported as a separate grammar condition.

9 Scheduling, memoization, and relative completeness

9.1 A portfolio with a fair reference lane

A reasonable initial portfolio contains a native focused lane, a library-composition lane, a structural-recursion lane, a behavioral/domain lane, and a proof-obligation lane. Each exposes a bounded `step` operation returning progress, proposals, suspended work, or exhaustion at a stated bound. A lane must yield during expensive candidate generation, not merely between completed candidates.

Maintain two scheduling layers. A best-first layer prioritizes promising states. A fair reference layer enumerates increasingly costly admitted recipes and verification efforts. Fixed positive quanta for both prevent heuristic preferences from starving an otherwise reachable candidate. Work queues are allowed to be bounded in a practical mode, but dropping states must then be reported as an incompleteness event.

Costs should be vectors: construction size, eliminations, recursion depth, carrier complexity, equality transports, unresolved constraints, proof effort, and operational estimates. A scalar combination is an engineering heuristic, not an admissible A* heuristic unless that property is separately proved. Keep a Pareto frontier or diverse top candidates when different objectives conflict.

9.2 Budgets must include proving the candidate

Candidate generation and verification are two unbounded searches. A scheduler that fairly enumerates programs but permanently discards each after a fixed failed proof attempt is not relatively complete for certified synthesis. An unresolved candidate needs a verification continuation with increasing proof budget, unless a valid counterexample has ruled it out.

Likewise, a fixed display window of ill-behaved inhabitants must not masquerade as exhaustion of all useful implementations. The user may set a candidate or verification cap, but reaching it should produce `Unknown(candidateBudget)`, retaining the distinction between an algorithmic impossibility and a chosen resource policy.

Theorem 9.1 (Conditional relative completeness). *Fix an effectively enumerable grammar $\mathcal{G}$ of finite candidate-and-certificate recipes for a frozen problem. Assume that (i) every required finite validation step terminates when sufficiently provisioned, (ii) scheduling eventually executes every finite admitted recipe and gives its validation sufficient effort, (iii) the reference lane prunes only with a checked impossibility certificate or a success-preserving replacement by an admitted, retained recipe of strictly smaller natural-number rank, and (iv) no permanent resource cap discards the successful recipe. If an admitted recipe constructs $t : T$ and a required proof $p : \Phi(t)$, then the search eventually offers a certified candidate.*

Proof. Choose a finite successful recipe. Effective enumeration and fairness eventually schedule its finite prefix of construction steps. An impossibility certificate cannot exclude a successful recipe. Each permitted replacement preserves success for the frozen grammar, specification, and policy; strict rank decrease makes every replacement chain finite. Thus a successful representative remains available. The same fairness condition applies to its finite certificate and validation sequence. After those steps complete, the acceptance path checks the candidate and certificate and emits the result. $\square$

This is a design theorem under explicit assumptions, not a formal completeness proof of the included tactic. It also does not say that every inhabited Lean type has an inhabitant in a particular practical

grammar. An unrestricted reference enumerator may be conceptually useful, but the intended coverage comes from useful schemas and indexes, not from waiting for raw enumeration.

9.3 Cache keys and reuse rules

A cache key must include the frozen environment identity, abstracted ordered context, expected type, relevant specification, universe constraints, transparency policy, grammar version, local-instance identities, recursive permissions, and the applicable observation bank. Branch metavariable numbers are not stable canonical names; contexts and holes need an alpha-normalized representation with explicit dependencies.

Positive reuse is by instantiation and rechecking. A cached proof or term can be valuable even when the new query is slightly different, provided the engine constructs a checked transport or substitution. A negative entry is valid only at its stated grammar and resource frontier. “No solution within depth 6” is not a permanent fact about the type.

Do not merge two programs merely because they agree on current tests. They may differ on the next counterexample. Such agreement is useful for ranking and for retaining a compact set of representatives with recoverable alternatives. Destructive merging requires proved equivalence under the contexts in which the programs will be used, or an explicit loss of completeness. With dependent consumers, even a proved equality may require a transport; textual replacement is not enough.

9.4 State-selection policy

Among ready holes, prefer those constrained by rigid expected heads, concrete observations, or shared dependencies. Type arguments determined by result unification should be solved before an expensive instance search. A witness with a cheap contradictory postcondition should be tested early. An independent proof can run in parallel, but a proof that may determine a data hole remains part of that same branch transaction.

A useful empirical score is a weighted sum of outstanding computational obligations, estimated provider distance, unresolved dependent indices, and certificate cost, with small rewards for covered observations. The weights should be trained or tuned on held-out development tasks and logged. The fair lane is the safeguard against a score that consistently misjudges an important construction family.

10 Recursive programs as synthesized elimination schemas

10.1 The right search object is a schema, not unrestricted self-reference

A native engine can construct Lean recursors directly or elaborate ordinary recursive definitions through a tightly controlled adapter. It should support both representations internally: a recursor is a reliable semantic core, while equation-style source is usually more readable. Lean’s structural and well-founded recursive elaboration supplies the final legality check [9].

The search order should try existing library combinators, small eliminations, structural recursion, generalized structural helpers, and finally well-founded recursion. This is an ordering policy rather than a restriction that all useful algorithms are folds.

For a family $I(\vec{p}, \vec{i})$ and an input $x : I(\vec{p}, \vec{i})$, a recursive schema consists of a motive $M(\vec{i}, x)$, constructor branches, recursive hypotheses or permitted calls, and any termination obligations.

Motives are synthesized from the target and specification. Their indices and varying arguments must be generalized sufficiently for recursive calls to type-check.

A crucial prohibition is adding a freely callable local

$$f : \forall x, T(x)$$

as though the desired recursive function had already been proved total. That produces circular justifications and makes synthesis deceptively easy. A recursive call must be mediated by a capability recording why that particular call is allowed.

10.2 Recursive-call capabilities

A capability is an engine object, not an extra logical axiom. It records the recursive function being constructed, its parameter telescope, the current major premise, admissible smaller arguments, and the evidence or structural path authorizing each call. For a list branch $a :: xs$, a structural capability authorizes recursion on xs , not on $a :: xs$ again.

Applying a capability creates the appropriate native recursor argument or a recursive call inside a checked skeleton. It also instantiates the recursive specification at that smaller input. If a helper has an accumulator, the capability specifies which parameters may vary and which remain fixed. A call with a new accumulator can be structurally valid; a call with an unchanged list is not made valid by changing that accumulator.

For indexed families, the smaller argument carries its own index. The capability's type is not guessed by subtracting one from the outer index. It is obtained from the actual constructor field and the corresponding recursor information. This is particularly important for nested and mutually recursive inductives, where a hand-written “one smaller field” heuristic is insufficient.

10.3 Algorithm for structural schema discovery

The following algorithm is intended for the first substantial recursion implementation.

```
proposeStructuralSchemas(problem):
  for major in ranked recursive inputs:
    inspect the native inductive and recursor metadata
    choose parameters that remain fixed
    generalize indices and varying arguments needed by recursive calls
    construct candidate motives from target and local specification
    for each constructor branch:
      expose constructor fields with their actual dependent types
      register recursive hypotheses / structural-call capabilities
      restrict observations to this branch; simplify its specification
      create program holes and proof holes in one branch state
    search all branches with shared outer constraints
    elaborate/check the complete recursion skeleton
    export only after its type and required specification are checked
```

Candidate motives should begin with a small grammar: the target family, functions over unsolved trailing arguments, dependent pairs or subtypes carrying the postcondition, and tuples of these. Motive enumeration becomes expensive quickly, so the engine should retain a visible motive frontier and allow a user to supply an invariant or helper signature. A user-provided motive is valuable guidance, not evidence that the resulting branch proofs are automatic.

10.4 Indexed-vector map: a complete worked construction

Let `Vec A n` have `nil` and `cons` constructors, and suppose the desired program has type

```
(f : A -> B) -> {n : Nat} -> Vec A n -> Vec B n
```

The motive is $M(n, xs) = \text{Vec}(B, n)$. The `nil` branch requires $\text{Vec}(B, 0)$ and admits `nil`. The `cons` branch exposes $x : A$, $xs : \text{Vec}(A, n)$, and a recursive result $tail : \text{Vec}(B, n)$. The target is $\text{Vec}(B, n+1)$. Constructor application reduces it to a B and the required tail; application of f to x supplies the head.

The artifact tests this exact division of labor:

```
def vecMap (f : A -> B) : {n : Nat} -> Vec A n -> Vec B n
| _, .nil => by leant2_core
| _, .cons x xs =>
  let tail := vecMap f xs
  by leant2_core
```

The recursive skeleton and recursive call were supplied by the author. The tactic synthesized the branch contents. A separate, author-written proof establishes

```
(vecMap f xs).toList = xs.toList.map f
```

for every indexed vector. This demonstrates that native synthesis can operate within dependent recursive branches; it does not demonstrate automatic discovery of the motive or the recursion skeleton.

An instructive implementation correction arose in the experiment. A first expanded version of `Vec.toList` accidentally fixed an ambient index n across recursive calls. Changing its signature to an explicit `{n : Nat} -> Vec A n -> List A` generalized that index correctly. This is precisely the kind of distinction the production motive generator must track.

10.5 Simultaneous program and proof synthesis

Instead of constructing an output and only later proving its property, strengthen a recursive motive to

$$M(x) = \{y : T(x) \mid R(x, y)\}.$$

A recursive hypothesis now contains both a recursive result and its certificate. The branch can use the certificate to discharge local proof obligations while selecting the result. The resulting program can project the data component; the proof component is available for a final theorem.

For a filtering task, for example, a length-indexed output alone says too little. A useful specification includes that the output contains exactly the input elements satisfying the predicate, in their original order. The `nil` branch and the two `cons` branches then receive meaningful obligations. One branch prepends the current element; the other reuses the recursive result. Element-order reasoning, predicate evidence, and the new output length remain correlated.

This construction may be easier to discover than an unstructured final correctness proof because the recursive hypothesis already matches the branch's needed fact. It does not remove the need for theorem search: permutation, ordering, or arithmetic lemmas may still be required.

Proposition 10.1 (Certified structural-schema soundness). *Suppose a proposed recursive implementation is elaborated to a Lean recursor application, every branch has the corresponding motive instance,*

and the resulting declaration and claimed specification pass kernel checking in the frozen environment. Then the exported declaration has the requested type and specification, regardless of how the motive or branches were proposed.

Proof. The native recursor type supplies the induction/elimination principle. Its checked application combines the checked branch terms at that motive. The final declaration check and specification check establish the requested judgments. No soundness assumption about the search procedure is needed beyond faithful validation and export. $\square$

10.6 Accumulator introduction and invariant discovery

Naive structural recursion can be extensionally correct yet operationally poor. Tail-recursive reverse suggests a generalized helper

$$go(xs, acc), \quad go(xs, acc) = reverse(xs) ++ acc.$$

The outer function is $go(xs, [])$. The additional accumulator parameter belongs to a synthesized helper, not to a changed public type.

A concrete invariant-discovery lane can proceed in four stages. Extract recurring subexpressions in residual branch specifications; propose a helper with those quantities as parameters; form candidate invariants from the original postcondition, constructor equations, and a small vocabulary of library relations; then ask the proof service to validate branch preservation and the initial-state implication. Counterexamples eliminate invalid invariant templates. Failure to prove preservation leaves the template unresolved rather than establishing it false.

For sorting, a helper invariant might relate an accumulator to a processed prefix by permutation and ordering. For an indexed traversal, it may relate an accumulator length to an arithmetic expression in the input index. These are family-specific search templates. A useful implementation should expose them as plugins with documented schema grammars instead of promising an unrestricted invariant synthesizer.

10.7 Well-founded, mutual, and nested recursion

When structural descent is inadequate, propose a relation and a well-founded recursion schema. The first relations should come from an explicit grammar: a natural-valued measure, lexicographic products of such measures, and selected library well-founded relations. Each recursive call generates a decrease proof. The relation itself needs a well-foundedness proof.

The distinction between integer and natural arithmetic matters here. An unrestricted decreasing integer argument does not establish well-foundedness. Natural subtraction also requires care: $0 - 1 = 0$ is not smaller than zero. A candidate measure must be checked against the actual guarded branches, not justified by a visual impression of a decreasing expression.

Mutual recursion needs a joint call graph and a common termination argument, often over a tagged sum of the mutually recursive input spaces. Nested datatypes require the actual nested recursor or a verified derived schema. These are sensible later milestones, not details to be hidden behind a generic “case split” operation.

11 Behavioral synthesis as construction, not only filtering

11.1 The behavioral loop

The predecessor’s behavioral interface already distinguishes proved, falsified, and inconclusive assertions [3]. Leant2 should retain that distinction while feeding specifications into construction earlier.

```
certifyingSearch(problem):
  pending := native/schema/domain proposal streams
  unknownProofs := resumable candidate-certificate tasks
  bank := typed observations for the exact specification
  while a scheduled task can run:
    if task is a partial program:
      propagate types, specifications and observations into its holes
      enqueue refinements, never promoting a heuristic to a proof
    if task produces a complete candidate c:
      check its exact type and construction policy
      replay applicable observations; reject only on justified failure
      schedule proof of Spec(c), plus counterexample search
    if a proof of Spec(c) completes:
      freeze, kernel-check, audit and emit c with that proof
    if a checked counterexample for c completes:
      reject c; add the typed observation to the bank
    if neither completes within the current quantum:
      retain the task as unknown and increase effort fairly
```

The proof and counterexample tasks are complementary. Failure of one is not success of the other. An SMT solver’s `unknown`, a stuck evaluator, or a proof-service timeout must not be converted into a Boolean verdict about the candidate.

11.2 Logical decomposition and library contracts

Specifications can guide construction through checked decomposition. A conjunction creates two proof obligations about the same program. Universal quantification introduces symbolic inputs. A precondition supplies local hypotheses in that symbolic branch. Equalities with constructor-headed right-hand sides suggest output shape, while equations for an applied function constrain its body.

Library providers should optionally carry verified contracts. Suppose a provider g has a theorem

$$\forall x, P(x) \rightarrow Q(x, g(x)).$$

Using g can reduce a desired postcondition to $P(x)$ and an implication from $Q(x, g(x))$ to the requested property. This is a weakest-precondition-style decomposition only when the corresponding implications are proved or supplied as obligations. An informal annotation in a provider index is a search hint, not a theorem.

A contract that is too weak must not cause rejection of a correct provider. For instance, knowing only that list length is preserved is insufficient to prove reversal, but it does not imply the provider fails reversal. The branch may need a stronger lemma or direct unfolding.

11.3 Live examples and incomplete terms

Partial evaluation of a sketch can propagate examples to its holes. A constructor-shaped output observation constrains its fields. A lambda observation supplies an input/output pair for the body. A known branch condition routes an observation to one branch. Applications of known pure operations may transform observations through verified or explicitly heuristic inverse rules. Live bidirectional evaluation provides a useful research model for this approach [15, 16].

Several safeguards are essential. Evaluation blocked on a hole returns a suspended constraint, not a fabricated default value. An unknown conditional may require both possibilities or a symbolic path condition. Two holes can be correlated by one example; solving their induced examples independently may lose valid solutions or invent incompatible ones. Recursive observations may only unfold calls justified by the active schema and its recursion permissions.

For a higher-order input, a sample must include an actual admissible Lean function or a justified finite representation. A solver’s arbitrary function table is not a closed counterexample to a theorem about all natural-number functions unless it is reconstructed as a total function with the needed behavior. Likewise, testing a polymorphic program at `Nat` and `Bool` does not automatically prove a parametricity theorem for every Lean type.

11.4 Counterexample identity and replay

A counterexample record contains the exact specification identity, an admissible input or observation, any precondition/path evidence, the candidate identity against which it was validated, and the resulting refutation certificate or trusted-checker derivation. The reusable bank stores the *observation*, not the fact that an earlier candidate failed it. Each new candidate is evaluated against that observation afresh.

Dependent inputs complicate reuse. If an input contains a proof of a bound involving the candidate, a value valid for one candidate may not be a legal test for another. The replay layer must reconstruct the input in the new dependent context or mark the observation inapplicable. Counterexample shrinking must preserve typing and preconditions; an out-of-range index is not a smaller valid counterexample to a bounds-respecting program.

Proposition 11.1 (Sound counterexample pruning). *Suppose $\Phi(c)$ implies an observation $O(c, a)$ for an admissible a , and Lean checks a proof of $\neg O(c, a)$. Then c cannot satisfy Φ in the frozen context.*

Proof. Assuming $\Phi(c)$ gives $O(c, a)$ by the checked implication, contradicting the checked negation. The resulting proof has type $\neg\Phi(c)$. $\square$

A raw compiled evaluation can be an efficient scheduling signal. To justify logical pruning in a certified-completeness lane, its result must be reconstructed as a Lean proof, checked by a proved evaluator, or covered by an explicitly admitted trust policy. Performance shortcuts belong in the receipt, not hidden in the meaning of “checked.”

11.5 Proof services must not silently choose the program

A proof service receives an obligation, a budget, a permitted assignment set, and the frozen policy. In a read-only mode it may solve proof holes but not computational metavariables. In a synthesis-aware mode it may propose assignments to data holes, but those assignments must be returned as a whole branch refinement with backtracking information.

Without this boundary, a tactic might solve an existential by choosing an arbitrary witness and thereby commit the program search to an unintended implementation. Conversely, forbidding every data assignment would lose useful constructive proofs. Both modes are needed; their effects must be explicit.

The default service can combine bounded simplification, arithmetic reasoning, constructor and elimination rules, and optional Aesop or other project tactics. Stronger tools, including Forge or external proof search, can implement the same interface. The service reports solved, refuted, or unknown with proof terms where applicable. It does not bypass the common acceptance path.

12 Certified domain lanes and solver integration

12.1 A domain interface with a semantics

A domain plugin should define a finite or effectively enumerable syntax D , a denotation into the relevant Lean type, a well-formedness predicate, a proposal procedure, and certificate checkers. A positive result includes enough information to construct a Lean proof about the denoted program and then transport that proof to the exact original candidate.

The conceptual interface is:

```
DomainPlugin:
  recognize : FrozenProblem -> Option DomainProblem
  syntax, typing, denote
  enumerateOrSolve : DomainProblem -> Resumable ProposalStream
  checkPositive : proposal + certificate -> Lean proof or refusal
  replayNegative : model -> typed observation + refutation or refusal
  coverage : optional theorem relating domain and original problem
```

Recognition may be incomplete. A failed recognizer means that this lane does not apply. It is not a refusal by the whole native engine. A successful recognizer must retain the original expressions and a precise relation to the domain problem; an approximate translation can still guide search, but its approximation direction determines which conclusions are valid.

12.2 Suitable first domains

Good initial lanes are finite Boolean and bitvector functions, small affine arithmetic programs, constructor-bounded data transformations, and library-composition grammars with verified contracts. Each has a clear finite search space or a manageable certificate language. Natural-number arithmetic side conditions can be delegated to Lean proof-producing tactics rather than introducing a second trusted arithmetic solver.

Lists admit useful abstract domains such as length and element-membership summaries. These abstractions are best used to prioritize or refute candidates when the abstract conclusion is justified by a proved relation to concrete evaluation. Length equality cannot prove element order or a map's pointwise action. An overapproximation with no abstract solution can rule out concrete solutions only if the abstraction is proved to include every admitted concrete completion. A satisfiable abstract constraint may have no realizable concrete program.

12.3 External SMT and synthesis engines

An external solver may propose a candidate, a model, an invariant, or a proof certificate. A satisfying assignment can be reconstructed into a Lean term and checked. An unsatisfiable result is not by itself a Lean theorem: it needs proof reconstruction, a verified certificate checker, or a separately stated trust extension. The default should be reconstruction rather than enlarging the trusted computing base.

Semantics mismatches must be addressed at the adapter boundary. Lean natural subtraction is truncated; integer arithmetic is unbounded; bitvector arithmetic wraps; division and remainder have totalized corner cases; dependent indices carry typing obligations. A solver encoding that quietly changes these conventions can reject correct programs or certify an unrelated problem. Higher-order or quantified formulas outside the adapter’s proved fragment remain uninterpreted constraints or are refused by that lane.

The simplest initial external interface need not use a solver at all. The affine experiment in Section 15 is entirely Lean: a finite grammar, an executable observation checker, and a proved characterization of the universal specification. This demonstrates the domain architecture without assuming a general Lean-to-SMT translation.

12.4 Typed equality saturation and optimization

A future optimizer may maintain equivalence classes of candidate expressions, but equality evidence must be typed. A rewrite is an equality theorem instantiated at the actual arguments and universe levels; replacing a subterm uses a checked congruence or transport. Dependent consumers can require a changed type or explicit cast. Heterogeneous equality is not a license to replace any term with a similar-looking one.

Use normalization and proved rewrites to remove redundant constructor/projection pairs, beta-redexes, or unnecessary transports. Keep the specification proof associated with the transformed term, either by direct rechecking or by a proved equality transfer. Runtime improvement is a separate objective: a proof-preserving rewrite can still change allocation or asymptotic behavior in an unfavorable direction.

13 Worked scenarios and boundaries

13.1 Dependent construction with a delayed witness

For $\{n : \mathbb{N} \mid n = 1\}$, apply the subtype constructor and retain both the witness and equality proof goals. The witness alternatives 0, 1, and larger numerals share the equality continuation. The first is rejected when its proof fails; the second succeeds by reflexivity. In the prototype this is actual search, not a supplied witness.

For a dependent pair $(x : A) \times B(x)$, the same mechanism handles a field whose type is not known until the first field is chosen. The production engine should also try proof-driven witness determination: an equation may solve the witness metavariable before explicit enumeration. Both paths must retain the full continuation.

13.2 Higher-rank plumbing without an approximate rank calculus

To synthesize

```
((((X : Type u) -> X -> X) -> A) -> A)
```

introduce the outer function, apply it, and construct its polymorphic argument by introducing X and then a value $x : X$. The body returns x . The included `higherRank` test exercises this native route.

Across independent universes, the engine keeps u and v rigid and lets each provider occurrence instantiate its own levels. This is a representational advantage over carrying special-case universe metadata through another type language. It does not make invalid impredicative encodings valid or guarantee that search will select a particular desired use of both polymorphic inputs.

13.3 Certified sorting

A type such as `List A -> List A` admits many implementations. Add a relation specifying sortedness and permutation, with the required order and decidability assumptions already present in the query. A library lane can try an admitted sorting function and its theorem. A construction lane can propose insertion sort: recurse on the tail, synthesize an insertion helper, and use the recursive sortedness and permutation facts in its branch obligations.

If no order assumptions are supplied, the system must not invent them. It can report that a proposed schema requires a comparison provider. The user may explicitly change the query, but a result for that changed query is not an implementation of the original frozen one.

13.4 Quotients, extensional values, and effects

Quotient-valued constructions require representatives and respectfulness proofs where the eliminator demands them. Structures with function fields may require extensional equality theorems rather than finite testing. Arrays require both their data and bounds semantics; a conversion through lists can be a correct candidate, but its cost must not be confused with a native array algorithm.

Effectful goals need a distinct behavioral model. A type `I0 A` describes an action, not an automatically safe experiment to execute during search. Pure `State` or `Except` programs can be evaluated against an explicit interpreter and state specification. Real I/O requires a sandbox, a capability policy, and a semantic specification of permitted effects. The first release should synthesize such actions only from approved combinators and avoid running arbitrary proposed I/O as a test.

13.5 Constructive, classical, and executable modes

A proof-search mode may admit standard classical principles under an explicit axiom policy. An executable-program mode should not use noncomputable choice as an unnoticed implementation strategy. The logical inhabitance of a type and the availability of intended executable code are different deliverables.

An executable policy must examine transitive dependencies, not just the outer declaration keyword. A total-looking wrapper can call a partial implementation or an external primitive. Conversely, the search engine itself may use partial metaprograms without contaminating the generated total term. The

prototype’s `partial def solve` is an engine implementation choice; its synthesized object-language definitions are separately checked ordinary Lean definitions.

14 Acceptance, trust, and executable meaning

14.1 A small, explicit acceptance path

A candidate found by search is a proposal. Publication should pass through the following sequence.

1. Restore the exact frozen context and instantiate the candidate’s assigned metavariables. Reject unsolved expression or universe metavariables and escaped local variables. Internal level choices must not silently add new public parameters or identify originally independent ones.
2. Construct the closed declaration by abstracting the exact telescope, including the dependencies of local definitions. Freeze its type separately from its proposed body. Do the same for the behavioral theorem; its statement mentions this exact implementation.
3. Check all generated auxiliary declarations in dependency order in a disposable environment. Reject arbitrary new axioms and circular dependencies. Force ordinary kernel checking; a metaprogram’s successful type inference is not the acceptance criterion.
4. Audit the transitive axiom inventory and computational dependency policy. The logical and executable assessments are recorded separately.
5. Delaborate for display and export. Re-elaborate the exported source against the frozen public signature, or retain a kernel-checkable artifact and a precise source correspondence. Commit only the validated declarations and their receipts.

The API probe checked the relevant Lean 4.34.0 entry points: `MVarId.intro1`, `MVarId.cases`, `Meta.saveState`, `Kernel.check`, and `Kernel.Environment.addDecl`, all in the `Lean` namespace. A production adapter should encapsulate such APIs and be tested separately for each supported Lean version. `Meta.check` and `isDefEq` are useful during search, but do not replace the ordinary kernel declaration boundary [10].

Theorem 14.1 (Acceptance soundness, relative to Lean and the declared policy). *Assume the frozen environment is the environment against which exported declarations are checked, the kernel is not bypassed, and the acceptance implementation faithfully performs the preceding checks. Every certified result contains a term of the frozen requested type and, when required, a proof of its frozen behavioral specification. Its logical assumptions are exactly those exposed by the transitive dependency audit.*

Proof. The closed term declaration is accepted only with the frozen type. The certificate declaration is accepted only with the proposition obtained by applying the frozen specification to that term. Abstracting and later instantiating the recorded telescope yields the corresponding contextual judgments. Transitive dependency inspection records the axioms used by these declarations. The proposal mechanism does not occur as a premise of either typing judgment. $\square$

This theorem is an architectural argument, not a Lean formalization of the entire acceptance implementation. A private `VerifiedCandidate` wrapper around `Expr` values can enforce API discipline, but its name alone is not a formal typing theorem. Kernel-checked object-language proofs and carefully implemented boundary ownership are still required.

14.2 What a kernel proof does not certify

A proof is relative to its environment and local assumptions. If a user supplies an inconsistent axiom, a certified result can depend on that inconsistency. The system should expose such dependencies; it cannot turn relative provability into an absolute consistency guarantee.

Logical checking also does not certify every runtime implementation choice. External implementations, unsafe code, runtime overrides, and partial dependencies require a separate policy. A trusted compiled evaluator may be useful for testing, but a proof about a logical constant does not automatically validate an arbitrary foreign implementation of that constant. The reference documentation distinguishes Lean’s proof validation and runtime concerns [10, 11].

An initial executable policy should admit approved total computational declarations, approved runtime primitives, and proofs from an explicit axiom allowlist. It should reject or flag noncomputable choice used to obtain runtime data, unchecked external bodies, partial computational dependencies, and effects outside the allowed capability set. The exact runtime primitive allowlist belongs to the toolchain adapter.

14.3 Hardened verification for untrusted extensions

An in-process kernel call protects against ordinary mistakes in the search implementation, not against arbitrary hostile metaprograms sharing that process. Unreviewed plugins or generated metaprogram code should run in a sandboxed proposal worker. A clean verifier should receive only a bounded, validated declaration/term representation, reconstruct it against the independently frozen challenge, and avoid executing the proposal engine’s code. For higher assurance, add comparator-style statement matching and independent external checking, following Lean’s validation guidance [10]. Do not load attacker-provided compiled module structures as though they were harmless proof data.

The default native rule library can remain an ordinary reviewed component. The stronger boundary is needed when extension code is outside that trust assumption. The experiments in this article used ordinary remote compilation; they did not run comparator or an external checker.

14.4 No evidence laundering through presentation or caches

The identity of a candidate is not its pretty-printed string. Different universe instantiations, dictionary arguments, local contexts, or elaboration choices can display similarly. A proof or counterexample for one occurrence cannot be donated to another based on matching text.

Every receipt should identify the environment, problem, candidate, specification, admitted providers, and transformation history. An optimized candidate needs its own checked specification or a checked proof transfer from the old one. A cached result from another environment epoch must be replayed, not accepted because its hash is familiar. Hashes establish identity checks under an operational model; they are not proofs of semantic equivalence.

15 Compiler-checked experiments

15.1 Environment and reproducibility

Both final prototype files were submitted through the Wolfram connector to the AXLE check endpoint. The request selected `lean-4.34.0`, set `ignore_imports = false` and `theorems_only = false`, and

retained the files' sole import, `import Lean`. Each file reported Lean version 4.34.0. The source hash reported in Wolfram matched the corresponding local artifact, and the service's echoed source matched after trimming surrounding whitespace.

For both final requests, `okay` was true, `failed_declarations` was empty, and both Lean and service error lists were empty. The service emitted its two nondefault-header advisories. The core file also had one unused-variable linter warning in an expected-failure control. These warnings are retained in the evidence record rather than suppressed in the report.

There was no local Lean installation and no independent second-kernel replay in this environment. The provided source and toolchain pin make local replay possible, but it is not claimed to have happened. The service reports executor identities, not a Mathlib commit; no unstated Mathlib revision is claimed. The experimental sources themselves import only Lean.

Table 2: Final remote checks. Times are whole-file service measurements, not synthesis-only benchmarks.

	Native core	Affine behavior lane
File	<code>prototype/Core.lean</code>	<code>prototype/Behavior.lean</code>
Lean version	4.34.0	4.34.0
Remote status	Accepted; zero errors	Accepted; zero errors
Service total time	1,507 ms	1,387 ms
Request wall time	1,636 ms	1,516 ms
Cached response	No	No
Evidence	<code>evidence/Core.receipt.json</code>	<code>evidence/Behavior.receipt.json</code>

The two request identifiers are

```
527aff4b-47fd-4d50-b992-4d3d3d712b00
1caada9d-c739-46cb-a8e6-25a70b2b7335.
```

The JSON files are explicitly labeled selected-field transcriptions of the returned tool responses; they are not represented as byte-for-byte HTTP captures. A subsequent run with the included clients can retain the full response.

15.2 Native dependent search

The core tactic implements bounded continuation search with lambda introduction, local application, constructor application, reflexivity, and case splitting of nonrecursive, nonindexed local inductives. It uses native terms and saved states, not a translated approximation of the target type.

Thirteen declarations have bodies produced entirely by `leant2_core`: identity, composition, product swapping, sum elimination, dependent application, dependent-pair construction, a constrained natural-number witness, a two-universe plumbing task, a higher-rank callback task, false elimination, dependent-pair projection, strict-implicit identity, and extraction from a supplied class value. The indexed-vector map adds two synthesized branch bodies inside an author-supplied recursive sketch.

Table 3: What the core experiment establishes, and what it does not.

Mechanism	Checked artifact	Boundary
Dependent witnesses	<code>dependentChoice</code> , <code>sigmaRepack</code>	Small goals under depth bound 30; no general arithmetic synthesis claim
Dependent elimination	<code>dependentProjection</code>	Splitting a nonrecursive Sigma-like object; arbitrary indexed elimination is not implemented
Higher-order and levels	<code>compose</code> , <code>higherRank</code> , <code>twoUniverses</code>	Different from the predecessors’ original simultaneous two-box fixture
Binder and instance data	<code>strictIdentity</code> , <code>fromInstance</code>	Preserves strict implicit syntax and supplied class data; no general instance-search benchmark
Recursive data	<code>vecMap</code>	Skeleton and recursive call supplied; branch contents synthesized
Universal behavior	<code>vecMap_spec</code>	Author-written induction proof, not synthesized by the tactic
Negative controls	Expected failures on <code>False</code> , an impossible subtype, and <code>Nonempty A -> A</code>	Checks rejection behavior, not completeness or a proof that every such Lean type is empty
True noninhabitation	<code>uniformEmpty</code>	Author-written Lean contradiction from a hypothetical uniform inhabitant

Twenty named declarations in the core file were audited with `##print axioms`. Nineteen reported no axioms. The vector-map correctness theorem reported `propext`. In particular, the generated vector-map implementation itself reported no axioms. The four concrete observations in `allTests` were checked by `decide`; its audit was empty.

15.3 A finite certifying behavioral lane

The second prototype searches programs

$$f_{a,b}(n) = an + b, \quad a, b \in \{0, 1, 2, 3, 4\},$$

in lexicographic coefficient order. Its mandatory specification is

$$f(0) = 1 \quad \wedge \quad \forall n : \mathbb{N}, f(n + 1) = f(n) + 2.$$

The infinite quantifier is not replaced by a test bound. A Lean theorem proves, for arbitrary natural coefficients,

$$\text{Spec}(f_{a,b}) \longleftrightarrow (a, b) = (2, 1).$$

A certifier uses that theorem to return an optional subtype value carrying the universal proof. The theorem characterizes only the affine program language; it is not a general decision procedure for specifications of Lean functions.

The counterexample bank begins empty. The first candidate $(0, 0)$ fails the base observation. The next candidate that survives that observation, $(0, 1)$, fails the step observation at zero. After both observations are stored, later candidates can be cheaply rejected before the full certifier is invoked. The winner is $(2, 1)$.

The executed result and Lean-proved count theorem agree:

Grammar size	25
Candidates visited before success	12
Certifier invocations	3
Candidates pruned by the observation bank	9
Inconclusive cases in this run	0

Without bank pruning, visiting the same ordered candidates and invoking the certifier for each would require 12 invocations before that winner. This is an enumeration-count comparison, not a measured runtime speedup. The certifier in this microbenchmark is inexpensive and specialized.

The `replay_sound` theorem proves that a failed observation refutes the universal specification. The final winner has a proof-carrying type, and `run_result` checks its coefficients. All five audited behavioral theorems depend only on `propext` or on `propext` together with `Quot.sound`; none reported `sorryAx` or `Classical.choice`.

15.4 Failed attempts and prototype shortcomings

The history is not a clean-room claim that every attempt worked. An API probe used two incorrect names and was corrected. The first affine proof had a redundant `omega` after simplification had already solved the goal; Lean rejected it and the service refused evaluation of the resulting incomplete declarations. The expanded vector helper initially fixed its recursive index incorrectly. The final two files were separately checked after these corrections. A malformed compressed transport attempt failed its guard before any Lean submission. These events are summarized in `attempt-history.json`.

The core prototype is intentionally small. It has depth-first search, a depth bound of 30, no global library inventory, no fair resumable scheduler, no automatic recursion discovery, and no production exception classification. Its case splitter excludes recursive and indexed inductives. It neither optimizes programs nor automatically proves arbitrary behavioral properties. The affine prototype supplies a hand-proved specialized verifier and does not implement a general unknown-proof continuation manager. These limitations are specific implementation facts, not reasons to weaken the proposed architecture.

16 Evaluation: making practical coverage measurable

16.1 A benchmark with separate objectives

A useful benchmark distinguishes three questions: can the system inhabit the exact type, can it meet the intended behavioral specification, and can it produce acceptable executable code? A pass on the first is not a pass on the other two. Record all three, along with the admitted library and schema policy.

Build a stratified corpus from real Lean programming tasks, with synthetic boundary controls alongside it. The strata should include ordinary higher-order plumbing, dependent structures and subtypes, local typeclasses, native lists and trees, indexed vectors and finite indices, arithmetic side

conditions, structurally recursive functions, accumulator helpers, and selected well-founded algorithms. Proof-only tasks and effectful tasks should be reported separately from pure executable synthesis.

Every case freezes its Lean toolchain, imports, universe parameters, full statement, assumptions, mandatory specification, optional observations, provider exclusions, and resource settings. Exact legacy fixtures should remain exact: easier signatures, larger budgets, or supplied helpers are useful ablations, not replacements for the original task.

16.2 Preventing target leakage

When a declaration body is withheld as a synthesis target, remove access to that declaration, aliases revealing it, and the oracle implementation used to generate tests. If the benchmark permits a library implementation with an equivalent specification, classify the result as library reuse. For a from-primitives task, that implementation and its revealing wrappers must be excluded.

Splits should occur by task family or project component, not by trivial renaming of nearly identical declarations. Otherwise, a learned ranker can memorize patterns from the evaluation set. Hidden tests help detect overfitting, but a universal proof against the frozen specification remains the strongest behavioral criterion.

16.3 Baselines and ablations

The initial comparison should include the existing Leant/Djex route on its supported fixtures, a simple native constructor/application tactic, available Lean proof automation where applicable, and the proposed portfolio with selected lanes disabled. Use the same exact problem and document any features unavailable in a baseline rather than silently translating it to another task.

Ablations should isolate native representation from scheduling and synthesis power. Compare the native core with and without scoped context slices; with and without behavioral propagation; with supplied versus discovered recursive schemas; and with fair continuation scheduling versus a fixed candidate window. Record proof-service effort separately from candidate construction.

Useful metrics are the first certified result’s latency, verified success rate, unknown and refuted counts, explored refinements, normalization and unification work, peak memory, proof size, final checking time, and resulting program cost. Report medians and distributions over repeated measurements; do not infer end-to-end speed from one isolated elaboration timing.

16.4 Adversarial acceptance tests

The regression suite should actively try to defeat the boundaries. Include two local dictionaries with identical class types but different payloads; independent universe parameters; strict implicit binders; dependent fields sharing a witness; incompatible branch indices; a proposition whose proof cannot be eliminated into data; a candidate that passes every current example but fails a new one; and an inconclusive proof search that later succeeds with more effort.

Also test stale cache reuse after undo, worker restarts, source names shadowed by new declarations, timeouts during a supposedly safe normalization step, callbacks that try to assign ambient metavariables, generated auxiliary declarations leaking into later inventories, and exported source whose elaboration selects a different instance. The suite should contain actual false predicates, not only searches that happened to produce no candidates.

17 Implementation roadmap and acceptance gates

17.1 Milestone 1: a trustworthy native core

Implement the frozen problem, exact local-context capture, native branch ownership, continuation search, a single acceptance path, and ordinary Lean source export. Support introduction, local/provider application, constructors, nonrecursive cases, and a small proof service. Gate the milestone on dependent witnesses, independent universe parameters, strict implicits, local dictionary payloads, exact exports, and failure/timeout separation. The provided core is a feasibility seed, not the finished milestone.

17.2 Milestone 2: scalable compositional search

Add the declaration inventory, result-shape indexes, scoped demand graphs, parameterized recipe caching, and fair resumable scheduling. Instrument every pruning and resource event. Gate this stage on exact reproductions of the predecessors' difficult composition fixtures, with unchanged statements and clearly separated resource profiles. A new success at a larger bound is valuable but must retain the old miss in the ledger.

17.3 Milestone 3: dependent recursion

Implement automatic structural schemas for a selected set of ordinary and indexed inductives using native recursor metadata. Generalize varying indices and parameters, register recursive-call capabilities, and synthesize simple strengthened motives. Gate on map, append, length, index-safe access, and filtering with a relational specification. Distinguish supplied-sketch and skeleton-discovery results. Add mutual and nested schemas only when the simpler boundary tests are stable.

17.4 Milestone 4: certifying behavior

Integrate proof obligations with partial program search, typed observation propagation, a reusable counterexample bank, and an unknown-proof queue. Begin with small certified domains like the affine lane and finite Boolean grammars. Gate on universal specifications, proof-preserving candidate transformations, genuine counterexample replay, and correct handling of spurious abstract models. A bare external solver verdict must fail certification tests.

17.5 Milestone 5: broader algorithmic coverage

Add accumulator and invariant templates, selected well-founded measures, richer arithmetic and data-domain adapters, and stronger optional proof services. Introduce empirical program-quality optimization only after functional correctness and provenance are stable. Publish a held-out coverage dashboard with task strata, exact settings, and an explicit unknown category. This is the stage at which a claim of broad practical usefulness can become evidence-based.

17.6 Priority risks

The largest search risk is combinatorial explosion in motive, carrier, and provider selection. The response is demand-directed decomposition, reusable parameterized recipes, and fair fallback, not

arbitrary silent cutoffs. The largest semantic risk is treating dependent goals as independent or allowing a proof service to commit computational choices without backtracking. The response is branch ownership and continuation semantics.

The largest trust risk is accepting an artifact after checking a related but different expression, environment, or specification. The response is one frozen problem and one acceptance owner. The largest product risk is showing many trivial inhabitants while implying intended behavior. The response is separate certified, observed, and type-only result modes, plus visible specifications and operational objectives.

18 Conclusion

Leant2's opportunity is not merely to remove Haskell. It is to make Lean's dependent information operational throughout synthesis. Native types constrain candidate construction; specifications constrain holes before programs are complete; recursors organize both algorithms and their proofs; and solver or heuristic contributions remain proposals until checked.

The most important early engineering target is the interaction between these pieces. A sophisticated provider index is not enough if a failed proof cannot reopen a witness. A complete small calculus is not enough if its negative result is overinterpreted. A correct recursive skeleton is not enough if its indices were fixed at the wrong scope. The checked experiments make those boundaries tangible, while the proposed architecture provides a path from the small demonstrations to a practical Lean-native system.

A Implementation interfaces and invariants

The following interface is a *design sketch*. Only the two prototype modules are compiled experimental code. In a production API, constructors of accepted-result wrappers should be private to the acceptance module.

```

FrozenProblem:
  environmentIdentity, workerEpoch, toolchain
  universeParameters, exactLocalTelescope
  target : Expr
  mandatorySpecification : Option Expr
  observations : Array TypedObservation
  providerPolicy, executionPolicy, trustPolicy, resourcePolicy

RefinementProposal:
  problemIdentity, branchIdentity
  recipe : ImmutableRecipe
  affectedHoles : Array StableHoleId
  explanation : SearchDiagnostic

BranchState:
  nativeSnapshot
  rootTerm, pendingGoals, dependencyGraph
  delayedConstraints, recursiveCapabilities
  observationState, immutableRecipe

ProofServiceResult:
  solved(proofRecipe, declaredAssignments)
  refuted(refutationRecipe)
  suspended(continuation, unresolvedReasons)

SearchStepResult:
  progress(newContinuations)
  proposal(candidateRecipe)
  exhausted(exactGrammarFrontier, optionalCertificate)
  interrupted(reason, resumableState)

```

An `Expr`-typed field does not establish that an expression is well-typed. Each owner must document which native checks have been performed. A `TypedObservation` similarly carries a scope and checking provenance, not merely a printable input/output pair.

The minimum invariants are: all proposal effects remain branch-local until acceptance; all related dependent goals share one substitution state; every recursive use is justified by its schema; frozen assumptions and public universe parameters remain unchanged; resource consumption is never refunded by logical rollback; observations are never silently promoted to universal proofs; and export preserves the exact checked term/specification relationship.

For long-running work, each lane should expose a bounded transition rather than a function that promises to produce its next candidate eventually. A provider enumerator, proof searcher, or normalizer that does not yield defeats fairness at a higher scheduler layer. Heartbeat and process-level limits are fallback interruption mechanisms, not a proof of real-time responsiveness.

B The checked continuation-search implementation

This is the actual search procedure from `prototype/Core.lean`. Its dependence on native contexts and the whole remaining-goal continuation is visible in the code. The production extensions discussed in the article are intentionally absent.

```
partial def solve (fuel : Nat) (goals : List MVarId)
  (splitUsed : List FVarId := []) : MetaM Bool := do
  let goals ← goals.filterM fun g => return !(← g.isAssigned)
  if goals.isEmpty then return true
  if fuel == 0 then return false
  let g := goals.head!
  let rest := goals.tail!
  g.withContext do
    let attempt (step : MetaM (List MVarId))
      (used : List FVarId := splitUsed) : MetaM Bool := do
      let saved ← saveState
      try
        let children ← step
        if ← solve (fuel - 1) (children ++ rest) used then return true
      catch _ => pure ()
      saved.restore
      return false
    let target ← whnf (← g.getType)
    if target.isForall then
      if ← attempt (do let (_, child) ← g.introl; pure [child]) then return true
    if ← attempt (do g.refl; pure []) then return true
    let context ← getLCtx
    for localDecl in context do
      if !localDecl.isImplementationDetail then
        if ← attempt (g.apply (mkFVar localDecl.fvarId) {newGoals := .all}) then
          return true
    if let .const name _ := target.getAppFn then
      if let some (.inductInfo info) := (← getEnv).find? name then
        for ctor in info.ctors do
          if ← attempt (do
            let e ← mkConstWithFreshMVarLevels ctor
            g.apply e {newGoals := .all}) then return true
    -- Only non-recursive, non-indexed shapes are automatically split in this
    -- intentionally small prototype. Indexed recursion is tested as a sketch.
    for localDecl in context do
      if !localDecl.isImplementationDetail && !splitUsed.contains localDecl.fvarId then
        let ty ← whnf localDecl.type
        if let .const name _ := ty.getAppFn then
          if let some (.inductInfo info) := (← getEnv).find? name then
            if !info.isRec && info.numIndices == 0 then
              if ← attempt (do
                let branches ← g.cases localDecl.fvarId
                pure (branches.toList.map (·.mvarId)))
                (localDecl.fvarId :: splitUsed) then return true
    return false
```

The fuel is a recursive depth allowance, not a global bound on all explored alternatives. In particular, depth-first backtracking can explore many states at the same depth. Production scheduling must add a global charged-work ledger and make the continuation structure resumable. The prototype's catch-all exception handling is also not the production cancellation policy.

C The checked behavioral certificate boundary

The following excerpt is from `prototype/Behavior.lean`. The candidate returned by `certify` carries a proof of the universal specification. The counterexample theorem has a different role: it justifies rejecting a candidate that fails an observation.

```
def passes (c : Affine) (o : Observation) : Bool := decide (observe c.eval o)

theorem replay_sound (c : Affine) (o : Observation)
  (rejected : passes c o = false) : ¬ Spec c.eval := by
  intro spec
  have observed : observe c.eval o := by
    cases o with
    | base => exact spec.1
    | step n => exact spec.2 n
  have passed : passes c o = true := by simpa [passes] using observed
  rw [passed] at rejected
  cases rejected

def certify (c : Affine) : Option {c : Affine // Spec c.eval} :=
  if h : c = canonical then
    some ⟨c, (affine_spec_iff c).mpr h⟩
  else none

def counterexample (c : Affine) : Option Observation :=
  [Observation.base, .step 0].find? (fun o => !passes c o)
```

The complete file includes the proof of `affine_spec_iff`, the finite grammar, the counterexample-guided loop, the executed result, and the count theorems. It is deliberately a small self-contained certified domain, not a replacement representation of Lean types.

D Reproduction and artifact map

The archive contains this article’s modular LaTeX source and PDF, both final prototype files, a pinned `lean-toolchain`, a minimal Lake file, selected verification receipts, an attempt history, and the user’s supplied remote-checking tools. The failed indexed-helper revision is preserved only under `evidence`; it is not a build target.

From the `prototype` directory, a local replay with the specified toolchain is:

```
lake env lean Core.lean
lake env lean Behavior.lean
```

These commands are reproduction instructions, not a claim that a local replay occurred during preparation. For a remote replay using the supplied Python client, run from the archive root:

```
python tools/axle_check.py prototype/Core.lean \
  --no-audit --theorem Leant2Prototype.vecMap_spec \
  --environment lean-4.34.0 --timeout 40 \
  --output evidence/Core.replay.json

python tools/axle_check.py prototype/Behavior.lean \
  --no-audit --theorem Leant2Behavior.run_result \
  --environment lean-4.34.0 --timeout 40 \
  --output evidence/Behavior.replay.json
```

The files already contain their axiom diagnostics. The selected `-theorem` controls the client's principal audit target; all named audits remain available in the full response. The artifact checker also compares the source identities and all retained audit inventories in the existing receipts. It performs no new Lean proof check.

The two final source identities are:

```
Core.lean
1ff971316812ea9815096746deda2588f1a6cc03aedd232b773460a1d878ea7f

Behavior.lean
671b737e138efa949fb8334f94ae38827a3c3189362bb10e4b58ff42315e8535
```

These identify the submitted source bytes; they are not a substitute for checking the files. The returned source echoes were additionally compared modulo surrounding whitespace.

Build the article from its directory with `make`, or run `pdflatex` on `leant2.tex` three times to settle references and the contents. No external bibliography processor is required. The Python counterexample-count mirror and artifact-integrity checker can be run without Lean; their outputs are explicitly not Lean certification.

E Status ledger

Item	Status at delivery
Native continuation-search tactic	Implemented in Lean; final file remotely accepted
Thirteen small fully synthesized declarations	Remotely accepted at their exact included signatures
Indexed-vector recursive map	Supplied skeleton with synthesized branch bodies; accepted
Universal vector-map theorem	Author-written proof; accepted; propext dependency reported
Affine finite grammar and CEGIS loop	Implemented and executed in Lean
Affine universal certificate and replay theorem	Proved in Lean; accepted with the reported standard axiom dependencies
Global native scheduler and demand index	Architecture and algorithms proposed; not implemented here
Automatic recursion/motive/invariant discovery	Architecture and algorithms proposed; not demonstrated by the sketch test
General behavioral proof service or SMT adapter	Interface and algorithms proposed; not implemented by the affine microbenchmark
Original predecessor indexing/two-box fixtures	Reviewed as diagnostic evidence; not solved or rerun in this artifact
Large-scale practical coverage or speedup	Evaluation protocol proposed; no benchmark claim
Independent local kernel/comparator replay	Not performed; remote compiler checks and source identities retained

References

- [1] Vladimir Reshetnikov and contributors. *Leant: native-to-fragment translator*, `src/Leant/Synth/Fragment.hs`. Reviewed at commit `6bf05ad78c467989e68290f2d08bbbed40802d485`, September 2026. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/src/Leant/Synth/Fragment.hs>.
- [2] Vladimir Reshetnikov and contributors. *Leant synthesis internals*, `docs/synth-internals.md`. Same pinned Leant snapshot. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/synth-internals.md>.
- [3] Vladimir Reshetnikov and contributors. *Behavioral assertions during synthesis*. Same pinned Leant snapshot. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/behavioral-synthesis.md>.
- [4] Vladimir Reshetnikov and contributors. *Djex architecture*, `docs/architecture.md`. Reviewed at commit `e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5`. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/docs/architecture.md>.
- [5] Leant repository. *Leant/Djex Lean 4 Rewrite Analysis*, source under `docs/Leant_Djex_Lean4_Rewrite_Analysis`. Reviewed in the pinned September snapshot; the report itself analyzes earlier pinned revisions and Lean 4.33.0.

- https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/Leant_Djex_Lean4_Rewrite_Analysis/Leant_Djex_Lean4_Rewrite_Analysis.tex.
- [6] Leant repository. *Indexing composition after native kind adoption*. Diagnostic report, September 14, 2026, in the pinned Leant snapshot.
<https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/reports/2026-09-14-indexing-composition-diagnostics.md>.
 - [7] Lean development team. *The Lean Language Reference: Universes*. Online reference, accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/The-Type-System/Universes/>.
 - [8] Lean development team. *The Lean Language Reference: Inductive Types*. Accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/The-Type-System/Inductive-Types/>.
 - [9] Lean development team. *The Lean Language Reference: Recursive Definitions*. Accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/>.
 - [10] Lean development team. *The Lean Language Reference: Validating a Lean Proof*. Accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
 - [11] Lean development team. *The Lean Language Reference: Run-Time Code*. Accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/Run-Time-Code/>.
 - [12] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. *Program Synthesis from Polymorphic Refinement Types*. 2016 version of arXiv:1510.08419. <https://arxiv.org/abs/1510.08419>.
 - [13] Peter-Michael Osera. *Constraint-Based Type-Directed Program Synthesis*. 2019. arXiv:1907.03105. <https://arxiv.org/abs/1907.03105>.
 - [14] Jonathan Frankle. *Type-Directed Synthesis of Products*. 2015. arXiv:1510.08121. <https://arxiv.org/abs/1510.08121>.
 - [15] Justin Lubin, Nick Collins, Cyrus Omar, and Ravi Chugh. *Program Sketching with Live Bidirectional Evaluation*. ICFP 2020. DOI: 10.1145/3408991. arXiv:1911.00583. <https://arxiv.org/abs/1911.00583>.
 - [16] Niek Mulleners, Johan Jeuring, and Bastiaan Heeren. *Program Synthesis Using Example Propagation*. 2022 preprint, published at PADL 2023. arXiv:2210.13873. <https://arxiv.org/abs/2210.13873>.
 - [17] Jannis Limperg and Asta Halkjær From. *Aesop: White-Box Best-First Proof Search for Lean*. CPP 2023. DOI: 10.1145/3573105.3575671. <https://doi.org/10.1145/3573105.3575671>.